

Architecture Design Document

Mk1 system architecture: design-space exploration, selected architecture, interfaces and requirement allocation

Document	BOATY-ADD-001
Issue	Issue B (for review)
Date	28 September 2026
Status	AR-2 and CR-01 accepted by the owner
Inputs	SRS BOATY-SRS-001 Issue B; Concept Selection Report v1.1
Outcome	Architecture AR-2 'Smart bank': 8 subsystems, 22 interfaces, 184 requirements allocated

Document control

Issue	Date	Change	By
A	28 September 2026	First issue, for review.	Claude (drafted)
B	28 September 2026	Owner accepted AR-2 and CR-01 (cap £160, SRS Issue C). Weed-shedding mechanism refined during ICD work (bounded GUIDED velocity bursts); V-11 to V-13 added.	Claude, owner decisions

Role	Name	Signature / date
Author	Claude, for Project Boaty	
Reviewer / approver	Project owner	

Review guidance: section 3 is deliberately divergent, so challenge the options and the scoring. Section 4 onwards is the converged architecture that the ICD and subsystem specifications will build on. The decisions needing your input are summarised in section 10.

Contents

1. Introduction	4
1.1 Purpose	4
1.2 Approach: diverge, then converge	4
1.3 Related documents	4
2. Architecture drivers	5
2.1 Quality attributes, in priority order	5
3. Exploring the design space	6
3.1 Morphological chart	6
3.2 Five candidate architectures	7
3.3 Evaluation and selection	7
3.4 Focused trade studies	9
4. Selected architecture	12
4.1 Architecture principles	12
4.2 Physical architecture and subsystems	12
4.3 Software architecture	14
4.4 Network and link	15
4.5 Power architecture	16
4.6 Mode mapping (SRS → ArduPilot Rover)	17
4.7 Failsafe implementation	17
4.8 Mission pipeline	17
5. Interfaces	18
5.1 Interface register	19
6. Requirement allocation	21
7. Budgets	22

7.1 Mass	22
7.2 Power and endurance	22
7.3 Cost	23
8. Design decisions register	24
9. Assumptions to prove early, and risks	24
10. Decisions and next documents	25
Appendix A. Requirements by subsystem	26

1. Introduction

1.1 Purpose

This document defines the Mk1 system architecture. It covers the subsystems, how they connect, where each requirement is met, and why this architecture was chosen over the alternatives. It is the parent of the Interface Control Document (ICD) and the subsystem requirement specifications.

1.2 Approach: diverge, then converge

The concept report went from a concept to an outline architecture quickly. This document steps back first.

- **Drivers (section 2).** The requirements and quality attributes that actually shape the architecture.
- **Diverge (section 3).** A morphological chart of twelve design dimensions. Five deliberately different candidate architectures, scored against the drivers. Nine focused trade studies on the dimensions that matter most.
- **Converge (sections 4-8).** Principles, subsystems, software, network, power, modes, failsafes, interfaces, allocation and budgets. Then the design decisions and the assumptions to prove early.

Some things were already fixed and are not reopened: the catamaran concept, ArduPilot as the helm (SRS Issue B), the Pi 5, and the SRS itself. Everything else was open.

1.3 Related documents

Ref	Document
[1]	BOATY-SRS-001 System Requirements Specification, Issue B
[2]	Concept Selection Report v1.1
[3]	ArduPilot Rover documentation (ardupilot.org/rover) and MAVLink common message set (mavlink.io)
[4]	BOATY-ICD-001 Interface Control Document (next)
[5]	BOATY-SSS-xxx Subsystem specifications (next)

2. Architecture drivers

Most of the 184 requirements could be met by almost any architecture. These are the ones that can't:

Requirements	What they demand	Architectural question
SAF-001/002	Fence and failsafes on an independent RTOS processor, working with everything else dead.	Where safety logic may live
FS-001..013	Thirteen failsafe behaviours, most with timings.	Which layer implements each one
VAL-001..010	Every mission passes one deterministic validator, then adult approval, then read-back.	A single mission path
NLI-001..008	Spoken or typed instructions turned into missions via the Claude API.	Where language processing happens
SWE-001/003	All project code in Python. Helm behind a swappable interface.	Rules out C++ firmware of our own
COM-002/005	At least 100 m over water. Internet via phone. Boat link independent of internet.	Link topology
MC-001..015	Pi 5 bank station, big buttons, voice, daylight display.	Bank-side hardware
CHD-*, MEC-007/009	Tool-free chunky modules and a DUPLO deck for a 4-year-old.	Mechanical modularity
CON-001	Mk1 bill of materials $\leq$ £160 (target £150; raised from £120 by CR-01).	Pushes against everything above

2.1 Quality attributes, in priority order

When options conflict, the higher attribute wins:

#	Attribute	Meaning here
1	Safety and recoverability	The boat comes back, or stops inside the fence, whatever fails.
2	Simplicity	Fewest parts and processes that can be built, understood and debugged by one person.
3	Crew experience	It's his boat: buttons, voice, DUPLO, photos.
4	Cost	Close to the £100 target.
5	Extensibility	Room for duck spotting, a Python helm, longer range.

3. Exploring the design space

3.1 Morphological chart

Twelve independent design dimensions, each with the realistic options. Any combination of one option per row is an architecture. That's over 16 million, so the chart is used to build a few deliberately different candidates (3.2) and to focus the trade studies (3.4). The options selected in the end are shaded blue.

Dimension	Option 1	Option 2	Option 3	Option 4
Where the intelligence lives (plan, validate, UI)	On the boat	On the bank (Pi 5)	Split: bank plans, boat re-plans	In the cloud
Helm hardware	F405 flight controller (1 MB)	H743 flight controller (2 MB, Lua)	Pixhawk-class	Linux autopilot HAT
On-boat mission computer	None (FC + Wi-Fi bridge)	ESP32-S3 camera (C++)	Pi Zero 2W + camera	Pi 5 on the boat
Camera	Camera on mission computer	FC-triggered action camera	Live video, capture on bank	Old phone on the boat
Boat-bank link	Boat is Wi-Fi AP	Pi 5 is AP, antenna on 2 m pole	Travel router on the bank	868 MHz telemetry + Wi-Fi for photos
Internet on the bank	Phone Wi-Fi hotspot	Phone USB tether to Pi 5	LTE dongle	Plan at home only
Language → mission	LLM writes waypoints	LLM picks intent; Python makes geometry	Templates only	LLM in the loop during the mission
Speech	Cloud speech-to-text	On-device STT + TTS on Pi 5	Phone dictation	Typed only
Display	Screen built into the box	Phone / tablet web UI	Buttons and voice only	e-ink panel
Arming interlock	Software only	Waterproof toggle switch	Magnetic key switching motor power	Pull-pin
Hull construction	Printed foam-filled segments	PVC pipe pontoons, printed ends	Carved XPS foam, skinned	Printed monohull + outriggers
Battery	3S Li-ion 18650 + BMS	3S LiPo	10-cell NiMH	Two packs (motors / avionics)

3.2 Five candidate architectures

Each candidate takes a different position on the biggest question: where does the intelligence live, and how does it talk to the boat?

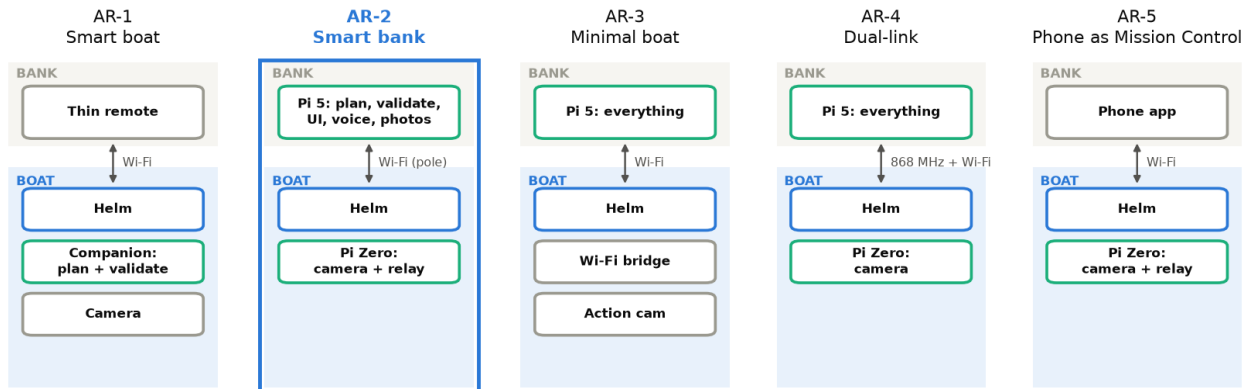


Figure 1. Candidate architectures. Blue = helm, green = where the project's intelligence runs.

ID	Idea	For	Against
AR-1 Smart boat	The boat plans and validates its own missions on a companion computer. The bank is a thin remote.	Keeps working if the link drops after launch.	Needs internet on the boat for language. More compute and power afloat. Hardest to debug. Most code on the water.
AR-2 Smart bank	The Pi 5 does language, planning, validation, UI and photo processing. The boat has the ArduPilot helm and a small Pi Zero 2W for camera and link.	Intelligence where power, internet and debugging are easy. The boat is simple. All project code is Python.	Costs more than the minimal boat. Photos depend on a second boat computer.
AR-3 Minimal boat	No mission computer. The FC plus an off-the-shelf Wi-Fi MAVLink bridge, with an FC-triggered action camera. The bank does everything.	Cheapest and fewest parts. No project code on the boat.	Photos only after the session, from the camera's SD card. No path to on-board duck spotting. Weak Wi-Fi bridge.
AR-4 Dual-link	AR-2 plus an 868 MHz telemetry radio for command and control. Wi-Fi only for photos near home.	Much longer control range; link loss rarer.	+£30 and two radios to integrate. UK 869 MHz duty-cycle limits. Range we don't need in a 100 m bay.
AR-5 Phone as Mission Control	A phone app replaces the Pi 5. The boat is as in AR-2.	Cheap and portable, with a screen, GPS and internet built in.	Touchscreen instead of big buttons. App code isn't Python. Ignores the owned Pi 5 (CON-002). Phone shared with a 4-year-old.

3.3 Evaluation and selection

Candidate	Safety	Link loss	Cost	Simplicity	Coverage	Python / ext.	Score
AR-2 Smart bank	5	4	4	4	5	5	4.55
AR-4 Dual-link	5	5	2	2	5	4	3.95
AR-3 Minimal boat	5	4	5	4	2	2	3.80
AR-1 Smart boat	4	4	3	2	3	4	3.40
AR-5 Phone as Mission Control	5	3	5	3	2	1	3.35
Weight	25%	15%	15%	15%	15%	15%	

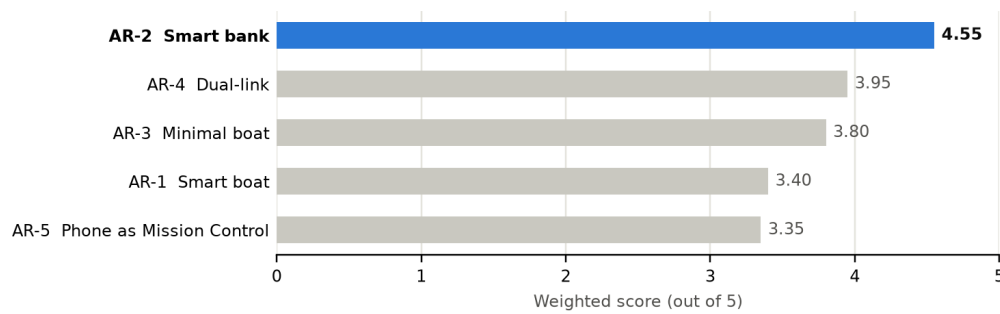


Figure 2. Weighted candidate scores.

AR-2 'Smart bank' is selected. It stays the winner when any single weight is doubled (saf: AR-2, link: AR-2, cost: AR-2, simp: AR-2, cover: AR-2, ext: AR-2). Its nearest rivals each win on one axis. AR-3 is cheaper, but gives up the live photo pipeline and the route to duck spotting. AR-4 has more range, which a 100 m home bay doesn't need; it remains the upgrade path if V-08 fails. Two ideas from the losing candidates are kept:

- From AR-3: no project code in the safety path. The boat's Python services can only make requests.
- From AR-1: the boat must carry on safely with the link gone. The loaded mission always ends in RTL, and the boat-side watchdog (B4) needs no bank.

3.4 Focused trade studies

T1. Turning words into missions

Option	For	Against
LLM writes waypoints	Maximum flexibility	Geometry mistakes; hard to test; many validator rejections
LLM picks an intent; Python planner makes the geometry	Language to the LLM, geometry to deterministic code. Testable. Landmarks like 'the island' work	Only understands intents we've built (the list grows)
Templates only	Simplest; offline	No 'just ask'
LLM in the loop during the mission	Adapts live	Needs link and internet mid-mission; no safety case

Decision: Intent + Python planner. Templates become pre-filled intents in the same pipeline, which is also the offline fallback (MIS-005).

T2. Boat-bank link and internet

Option	For	Against
Boat is the Wi-Fi AP	No bank hardware	Boat antenna is 0.3 m up in a box; Pi 5 would need a second radio for internet
Pi 5 is the AP, USB adapter on a 2 m pole; phone USB-tethered	Antenna height fixes the over-water Fresnel problem (section 4.4). Internet and link are separate (COM-005)	A pole and an adapter to carry (+£20)
Travel router on the bank	Good radio	Another box, battery and config
868 MHz telemetry + Wi-Fi	Long range	Cost, duty cycle, two radios

Decision: Pi 5 as AP with a pole-mounted USB adapter; phone USB tether for internet. Closes TBD-04. The pole is bought only if the first lake range test fails with the adapter at box height (COM-002).

T3. Helm hardware

Option	For	Against
F405 flight controller, 1 MB flash (~£25)	Cheap, common, ArduPilot Rover builds exist	1 MB builds leave out Lua scripting and some features
H743 flight controller, 2 MB (~£45-50)	Full feature set, Lua	+£20-25
Linux autopilot HAT	Everything on one Pi	General-purpose OS: fails SAF-001

Decision: F405-class, if the chosen board's firmware has every feature in V-01. Behaviours that would need Lua are done in Python on the mission computer instead (weed-shedding, link watchdog). Fallback: an H743 board, same wiring.

T4. Mission computer on the boat

Option	For	Against
None	Cheapest	No live photo pipeline; no DET-003 path
ESP32-S3 camera board	£14, low power	Firmware would be C++: conflicts with SWE-001
Pi Zero 2W + 5 MP camera	Python, picamera2, mavlink-router; can grow into duck spotting	+£9; Linux SD-card care needed
Pi 5 on the boat	Powerful	3-7 W and heat in a sealed box

Decision: Pi Zero 2W. This resolves the SWE-001 conflict found during SRS review.

T5. Hull construction

Option	For	Against
Printed foam-filled segments	Modular, child-assemblable, printable	Print time; seams
PVC pipe pontoons	Cheap, very robust	Less for the crew to build; fixed length
Carved XPS foam, skinned	Light	Messy; not modular
Monohull + outriggers	Different look	Worse camera platform; more parts

Decision: Printed segments, as in the concept. PVC pontoons are the fallback if printing proves troublesome. The foam fill makes seams non-critical.

T6. Speech in and out

Option	For	Against
Cloud speech-to-text	Accurate	Child's voice leaves the device; needs internet
On-device STT (Whisper-class) + TTS (Piper-class) on the Pi 5	Private, offline, free	A few seconds' latency; child speech accuracy to prove
Phone dictation	Easy	Goes to a third party; phone in child's hands

Decision: On-device. Closes TBD-05. Adult approval (VAL-008) catches mis-hearing.

T7. Battery

Option	For	Against
3S Li-ion 18650 pack + BMS	Hard cells, safer in a sealed box, cheap, ample current	Heavier per Wh than LiPo
3S LiPo	Light, high current	Soft pouch; puncture/swell risk; current not needed
NiMH	Very safe	Heavy, low energy

Decision: 3S Li-ion 18650 with BMS. Closes TBD-07.

T8. Arming interlock (step 1 of MOD-003)

Option	For	Against
Software only	No parts	Doesn't meet MOD-003
Waterproof toggle	Simple	A gland and a switch a child can flick
Magnetic key switching motor power	No hole in the box; the adult holds the key; cuts motor power independently of software	Reed switch + MOSFET switch to build

Decision: Magnetic key. The helm senses motor-rail voltage, so arming is refused while the key is out.

T9. Display

Option	For	Against
Screen built into the box	Always there	Glare, cost, power
Phone / tablet web UI served by the Pi 5	Any device, daylight brightness, no extra cost	Needs a clamp

Decision: Web UI on the operator's phone (already USB-tethered) or a tablet clamped to the box. Closes TBD-06.

4. Selected architecture

4.1 Architecture principles

#	Principle	Meaning
1	Safety lives in the helm.	Fence, failsafes and motor authority are ArduPilot's, on its own RTOS processor. Nothing we write can disable them while armed.
2	Intelligence lives on the bank.	Language, planning, validation, UI and photo analysis run on the Pi 5, where power, internet and a keyboard are.
3	The boat carries only what it must.	Helm, camera, link relay and small watchdogs.
4	Every link can fail.	The loaded mission always ends in RTL. Losing Wi-Fi, internet or a computer degrades function, never safety.
5	Words become intent; code makes geometry.	The LLM proposes, the planner draws, the validator checks, the adult approves, the helm enforces.
6	Python everywhere we write code; one simulator for everything.	The same Mission Control code drives SITL and the real boat.

4.2 Physical architecture and subsystems

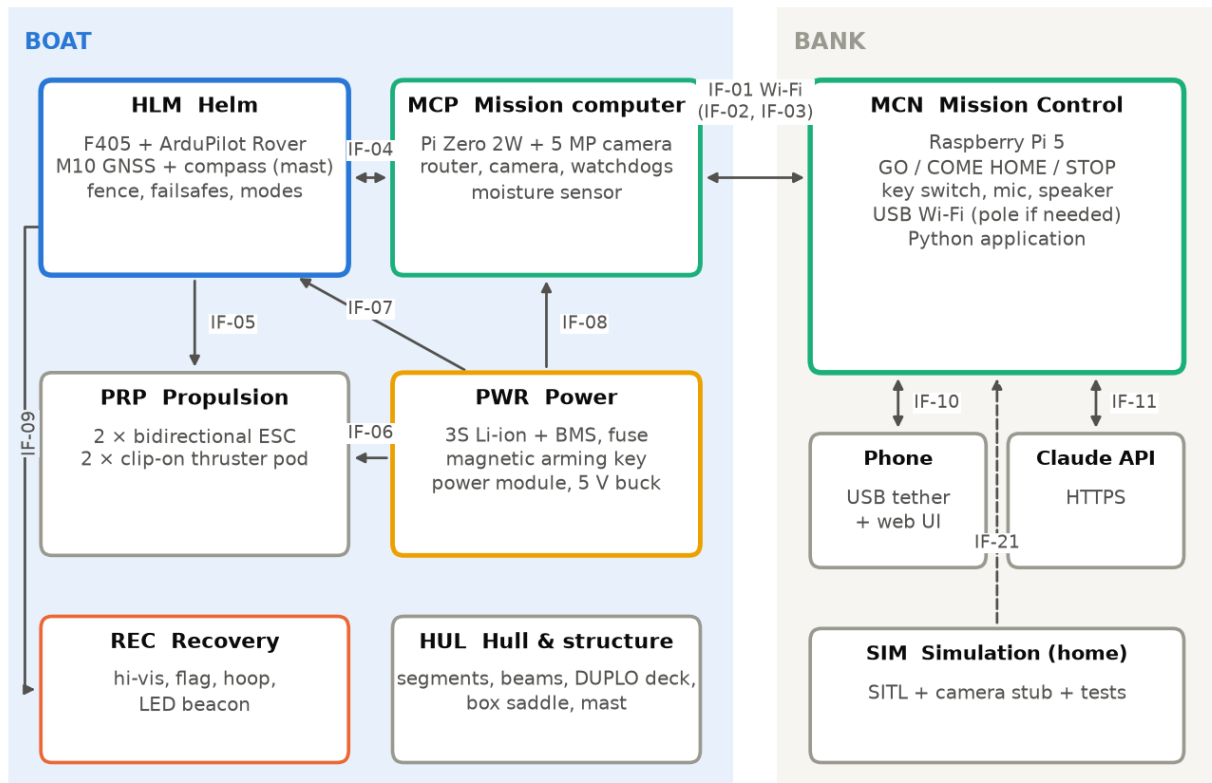


Figure 3. Physical architecture: 8 subsystems and the main interfaces (IF-xx, section 5).

Code	Subsystem	Where	Contents
HUL	Hull & structure	Boat	Six foam-filled printed hull segments, two crossbeams with M3 rail, DUPLO deck plate, electronics-box saddle, mast and sockets, carry handle.
PRP	Propulsion	Boat	Two clip-on thruster pods (submerged brushless motor, printed prop, finger/weed guard, strut, dovetail), two bidirectional ESCs, keyed waterproof pod connectors.
PWR	Power	Boat	3S Li-ion 18650 pack with BMS, main fuse, main switch, magnetic arming key and motor-rail MOSFET switch, power module (V/I sense), 5 V buck for the mission computer, harness.

Code	Subsystem	Where	Contents
HLM	Helm	Boat	F405-class flight controller running ArduPilot Rover (boat frame), M10 GNSS + compass on the mast, controlled parameter set, fence and mission storage, notify-LED output for the beacon.
MCP	Mission computer	Boat	Pi Zero 2W, 5 MP camera, microSD, moisture sensor. Python services: MAVLink router, camera, photo server, link watchdog, weed-shedding, health.
MCN	Mission Control	Bank	Raspberry Pi 5, USB Wi-Fi adapter (pole-mountable), GO / COME HOME / STOP buttons, adult key switch, USB mic, speaker, status LEDs, power bank, case, phone/tablet display. Python application: session manager, web UI, voice, planner, validator, helm interface, photo sync, captain's log, site store, logger.
REC	Recovery & signalling	Boat + bank	Hi-vis finish, flag, recovery hoop, LED beacon (driven by HLM), 'if found' label, bank recovery kit (pole + net, casting rod), optional tracker tag.
SIM	Simulation & test	Home	ArduPilot SITL (Rover, boat), simulated mission computer (camera stub serving test images), scenario runner (pytest), CI on push.

The LED beacon is driven by the helm, not the mission computer (DD-11). A boat with a crashed Pi Zero still shows its state.

4.3 Software architecture

Two Python applications and one third-party firmware. The Mission Control application does the thinking. The boat services are small and single-purpose. ArduPilot has the final say.

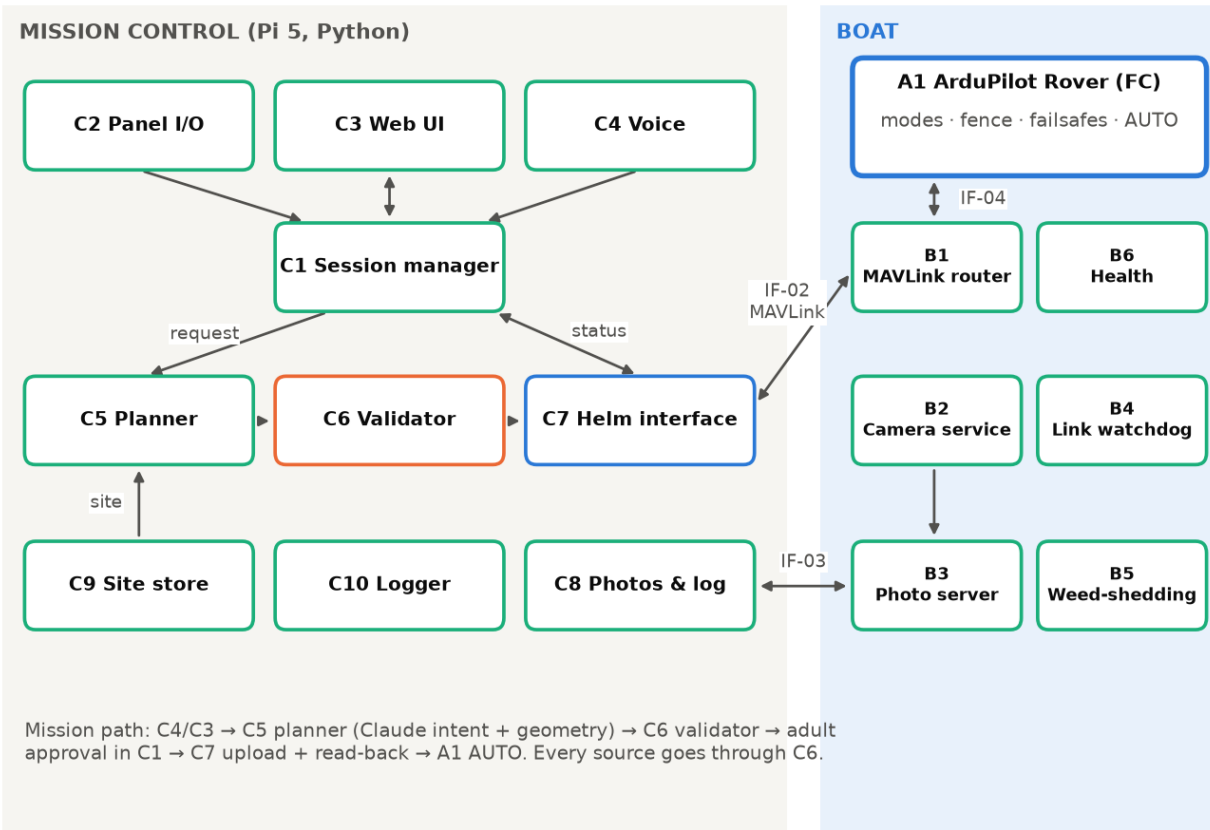


Figure 4. Software components and data flows. Orange = the validator gate; blue = the helm.

ID	Component	Responsibility	Key reqs
C1	Session manager	Session state machine; gates GO; checklist; arming sequence	MC-003, VAL-008/009, PRE-006
C2	Panel I/O	GPIO buttons, key switch, LEDs, press-and-hold	MC-002/004/008
C3	Web UI	Map, plan preview, status, checklist, fence editor, photo review	MC-005/006/009/015, FEN-002
C4	Voice	On-device STT (push-to-talk) and TTS announcements	NLI-002, MC-007
C5	Planner	Claude client (intent schema) + geometry generator + templates	NLI-003..006, MIS-001..006
C6	Validator	Pure, deterministic mission checks	VAL-001..007
C7	Helm interface	Helm API; ArduPilotHelm over MAVLink (pymavlink)	SWE-002/003, VAL-010
C8	Photos & log	Photo sync, waterfowl finder, captain's log	CAM-007, DET-001/002, LOG-004
C9	Site store	Fences, exclusions, landmarks, home points (GeoJSON, in git)	FEN-001/002, OPS-005/006
C10	Logger	Structured session logs, LLM I/O, approvals	LOG-002/003
B1	MAVLink router	FC UART ↔ UDP to Mission Control; local endpoint for B2-B6	COM-003, SAF-003
B2	Camera service	Capture policy (interval, photo points, command); geotags from MAVLink	CAM-001..004
B3	Photo server	Photo index and download over HTTP	CAM-007
B4	Link watchdog	No Mission Control heartbeat for 60 s in AUTO → command RTL	FS-003

ID	Component	Responsibility	Key reqs
B5	Weed-shedding	On stuck → up to 3 bounded GUIDED reverse bursts, then resume or stay in HOLD	FS-006
B6	Health	Moisture → RTL + alarm; temperature; storage	FS-010, PWR-010, CAM-004
A1	ArduPilot Rover	Navigation, modes, fence, failsafes, logging (third-party firmware + our parameters)	NAV-*, FEN-*, FS-*, LOG-001

Rule for boat-side Python (B4-B6): these services may only request safer states through MAVLink: HOLD, LOITER or RTL. The one exception is B5's weed-shedding. It may use GUIDED for bounded reverse bursts (at most 0.5 m/s astern, 2 s each, 3 in total), and must then hand back to HOLD. V-11 confirms the helm stops if B5 dies mid-burst. These services never arm, never change the fence or parameters, and never command AUTO or MANUAL (FS-007, SAF-003).

4.4 Network and link

The Pi 5 runs the Wi-Fi access point, using a USB adapter with an external antenna. The boat's Pi Zero joins it as a client and relays MAVLink between the helm and Mission Control. The operator's phone is USB-tethered to the Pi 5. That single cable gives internet for the Claude API and a route for the phone's browser to reach the web UI. Losing internet affects only planning (COM-005, MC-011).

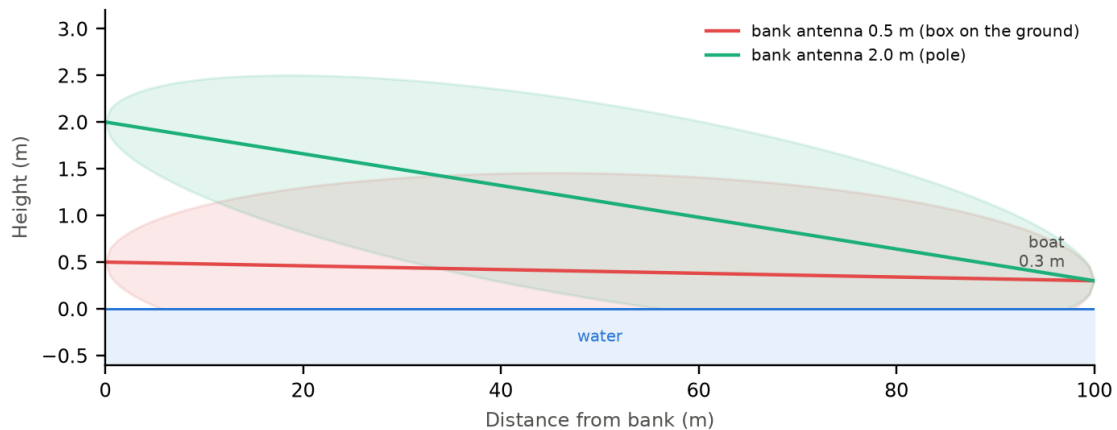


Figure 5. Why antenna height matters over water. The shaded bands are 60% of the first Fresnel zone (1.06 m radius at mid-path for 100 m at 2.4 GHz). Where a band dips into the water, reflections cost signal.

Link budget (100 m / 200 m)	Value
Free-space path loss, 2.44 GHz	80 dB / 86 dB
Pi Zero 2W transmit (typ.) + antenna	≈ +13 dBm, 0 dBi
Bank antenna gain	≈ +5 dBi
Received (free space)	≈ −62 dBm / −68 dBm
Needed for a low-rate link	≈ −85 dBm
Margin before Fresnel / reflection losses	≈ 23 dB / 17 dB

There's plenty of free-space margin. The real risk is the over-water geometry, hence the pole option (V-08).

4.5 Power architecture

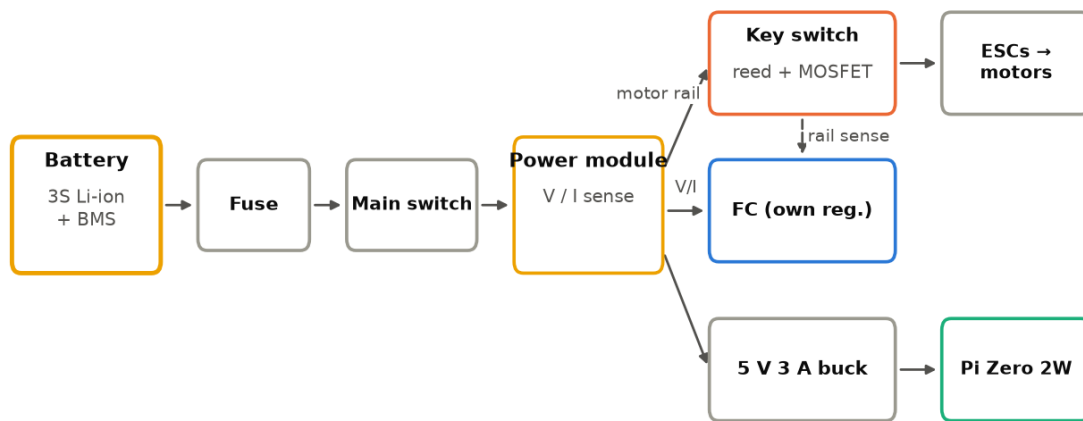


Figure 6. Boat power distribution. The magnetic key removes motor power independently of all software. The helm senses the rail and refuses to arm without it.

4.6 Mode mapping (SRS → ArduPilot Rover)

SRS mode	ArduPilot mode	Notes
DISARMED	Disarmed (any mode)	Motors inhibited; key out → motor rail unpowered.
HOLD (station-keeping)	LOITER	Boats hold position within the loiter radius.
HOLD (motors off)	HOLD	After STOP, position loss or stuck. Boat drifts.
MANUAL	STEERING (speed-controlled)	Speed limit applies; fence enforcement in this mode to confirm (V-02).
AUTO	AUTO	Validated mission; last item is RTL.
RTL	RTL	Ends loitering at home; to confirm (V-04).

4.7 Failsafe implementation

Where each failsafe lives. 'Native' means an ArduPilot parameter on the independent helm. 'Python' means a boat-side service, used only where ArduPilot on a 1 MB board can't do it and the requirement isn't the last line of defence.

Req	Failsafe	Layer	Mechanism	Type
FS-001	Low / critical battery	HLM	Battery failsafe (two thresholds)	Native
FS-002	Link loss, MANUAL	HLM	GCS failsafe	Native (V-03)
FS-003	Link loss, AUTO	HLM + MCP	GCS failsafe set to continue in AUTO; B4 watchdog commands RTL at 60 s	Native + Python
FS-004	Position loss	HLM	EKF failsafe → HOLD	Native
FS-005	Stuck	HLM	Crash check → HOLD	Native (V-05)
FS-006	Weed-shedding	MCP	B5 bounded GUIDED reverse bursts via MAVLink (V-11)	Python
FS-007	Mission computer down	HLM	Mission continues; nothing depends on the companion	By design
FS-008	Helm down	PRP	ESC stops on signal loss	ESC firmware (V-09)
FS-009	STOP	MCN → HLM	Mode HOLD (+ disarm) over MAVLink	Native command
FS-010	Water ingress	MCP → HLM	B6 commands RTL	Python
FS-011	Priority	HLM	ArduPilot failsafe ordering + design rule for B4-B6	Native + rule
FS-012	Log & announce	HLM + MCN	Helm log; C10 + C4 announce	Native + Python
FS-013	No single failure leaves fence	System	FMEA + SITL fault injection	Analysis

4.8 Mission pipeline

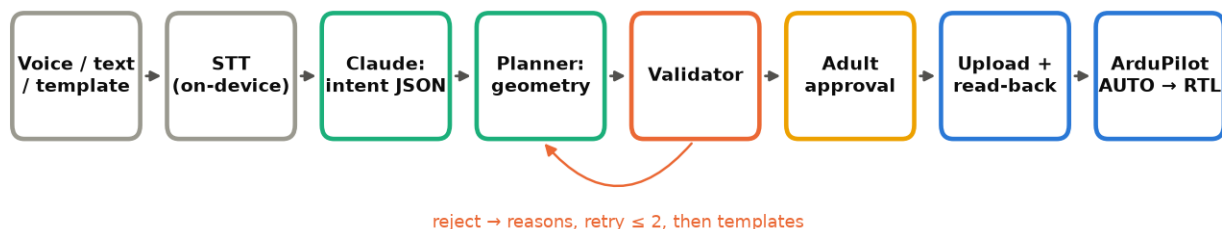


Figure 7. From words to motion. Every mission source (voice, text, template, map editor) enters before the planner or validator. Nothing bypasses the validator (VAL-001).

- **Intent, not waypoints.** The Claude API returns a small JSON intent, e.g. `explore(area='home bay', coverage='medium')`, `photo_stops(n=3, near='island')`, `return_home()`. The request uses tool use, so the output must match the schema (NLI-004).

- **Planner.** Deterministic Python turns the intent into geometry, using the site store's fence, exclusions and named landmarks: survey lanes, photo points and legs.
- **Native execution.** The mission becomes standard ArduPilot mission items (waypoints, timed loiters at photo points, final RTL) and runs in AUTO. The mission computer takes photos by watching mission progress over MAVLink. Nothing streams live steering, so the boat never depends on a companion to finish (DD-05).

5. Interfaces

The N² diagram shows which pairs of elements interact, with subsystems and externals on the diagonal. The register defines each interface to the level needed to scope the ICD.

HUL 16	17	18	19							20
17	PRP	06	05							
18	06	PWR	07	08						
19	05	07	HLM 22	04	02	09				
		08	04	MCP	01 03					
			02	01 03	MCN 13,14		15 21	10	11	12
			09			REC				
					15 21		SIM			
					10			Phone		
					11				Claude	
20					12					People

Figure 8. N² diagram (interface numbers; diagonal entries are interfaces internal to a subsystem).

5.1 Interface register

ID	Interface	Between	Type	Medium / protocol	Content	Reqs
IF-01	Boat-bank radio link	MCN ↔ MCP	Data	Wi-Fi 2.4 GHz 802.11n, WPA2, Pi 5 is AP	Carries IF-02 and IF-03	COM-001..006
IF-02	Command & telemetry	MCN ↔ HLM	Data	MAVLink 2 over UDP, relayed by B1	Modes, arming, mission upload and read-back, fence, status ≥ 1 Hz	COM-003, VAL-010, MOD-006
IF-03	Photo & health service	MCN ↔ MCP	Data	HTTP/JSON over IF-01	Photo index and files, capture commands, health	CAM-007, FS-010
IF-04	Companion link	MCP ↔ HLM	Data	MAVLink 2 over UART (FC TELEM port)	Router traffic, position for geotags, watchdog and weed-shedding commands	FS-003/006, CAM-003
IF-05	Motor commands	HLM ↔ PRP	Data	2 × DShot or PWM (bidirectional)	Left/right thrust ±100%	NAV-001, FS-008
IF-06	Motor power	PWR ↔ PRP	Power	11.1 V nominal via key-switched MOSFET, XT30	≤ 20 A per ESC	PWR-003/008, MOD-003
IF-07	Helm power & sensing	PWR ↔ HLM	Power	Power module: supply + analogue V/I; motor-rail sense	Battery V, I, mAh; key state	PWR-005/006, PRE-003
IF-08	Mission computer power	PWR ↔ MCP	Power	Dedicated 5 V 3 A buck	Isolated from motor transients	PWR-005
IF-09	Beacon drive	HLM ↔ REC	Data	Notify-LED output (WS2812-type)	Armed / failsafe / low-battery patterns	REC-004
IF-10	Internet uplink	MCN ↔ Phone	Data	USB tethering (RNDIS/NCM)	HTTPS to Claude API	COM-005
IF-11	Claude API	MCN ↔ Claude API	Data	HTTPS, Messages API with tool use	Instruction + site context → intent JSON; photos → captions (opt-in)	NLI-003/004, LOG-004
IF-12	Crew & operator panel	MCN ↔ Operator & crew	HMI	3 buttons, key switch, LEDs, mic, speaker, web UI	Commands, approvals, announcements	MC-002..008, CHD-005
IF-13	Mission format	MCN ↔ MCN	Software	JSON schema v1 (intent → mission → MAVLink items)	Contract between planner, validator and helm interface	MIS-004, VAL-*
IF-14	Helm API	MCN ↔ MCN	Software	Python abstract class	arm, disarm, mode, upload, read back, start, hold, rtl, stop, fence, status	SWE-002/003
IF-15	Site data	MCN ↔ SIM	Software	GeoJSON in git	Fence, exclusions, landmarks, home points	FEN-001/002
IF-16	Hull segment joint	HUL ↔ HUL	Mechanical	Flange + printed thumb-screws	Alignment, load path, tool-free	MEC-007, CHD-004
IF-17	Pod mount	PRP ↔ HUL	Mechanical	Dovetail + retaining pin; keyed 3-pin waterproof connector	Thrust load, alignment, swap < 2 min	MEC-008
IF-18	Box & glands	PWR ↔ HUL	Mechanical	Saddle on rail; PG7 glands	Sealing, cable entry, access	MEC-012
IF-19	Mast socket	HLM ↔ HUL	Mechanical	Socket on rail; GNSS ≥ 150 mm from power wiring	GNSS/compass, flag, beacon, hoop loads	MEC-013, REC-005

ID	Interface	Between	Type	Medium / protocol	Content	Reqs
IF-20	DUPLO deck	HUL ↔ Operator & crew	Mechanical	DUPLO-compatible stud grid	Grip / release force	MEC-009
IF-21	Simulation link	SIM ↔ MCN	Data	MAVLink 2 UDP from SITL; camera stub with the IF-03 API	Same traffic as IF-02/03	SWE-004/005
IF-22	GNSS & compass	HLM ↔ HLM	Data	UART (u-blox) + I2C compass	Position, time, heading	PRE-001, NAV-008

Every interface gets an ICD section with: owner, message/signal/geometry definitions, timing, error handling and a verification method. MAVLink interfaces (IF-02, IF-04) define only the subset used; the standard is referenced, not copied.

6. Requirement allocation

All 184 SRS requirements are allocated to one **primary** owner (bold), which is responsible for meeting and verifying it. Some also have **supporting** subsystems (+n) that must provide something for it. SYS = verified only on the integrated system. OPS = met by procedure. SAF-005 is not applicable (Route A). The build script checks that nothing is left unallocated.

Area	HUL	PRP	PWR	HLM	MCP	MCN	REC	SIM	SYS	OPS
MOD			1 +1	5 +2		2 +4				
PRE			+2	6 +1	+1	2 +4				+1
NAV		1 +2		7 +1						
FEN				5		2 +5				
FS		1 +1		8 +3	2 +2	1 +1			1	
SAF				3 +1		2 +3			1	
MIS				+6		6				
NLI						8				
VAL						10				
CAM	1 +1				7 +1	+2				
DET				+1	2	2				
MC				+1		15				
COM				1	+5	6				
REC	1			+1			6			
MEC	10 +2	3	+1	1					1	
PWR	+1	+1	9 +1	+3	1 +1				1	
SWE				+1	+2	4 +1		3	2	
LOG				1	+1	4				
ENV	3 +1	2	+1	+1	+1				1	
CHD	4	+4	+1			1			1	
OPS						+1				12
CON						+1			4	
Primary	19	7	10	37	12	65	6	3	12	12

Table: requirements per area and subsystem. The per-subsystem lists in Appendix A are the starting content of each subsystem specification.

7. Budgets

7.1 Mass

Item	g
Hull segments + foam (6)	500
Crossbeams (2)	120
DUPLO deck plate	60
Electronics box, tray, glands	150
Mast, GNSS, flag, hoop, beacon	110
Flight controller	12
Pi Zero 2W + camera + SD	30
ESCs (2)	30
Thruster pods (2)	150
Battery 3S 18650 + BMS	160
Power module, buck, key switch	30
Wiring and connectors	60
Fasteners	50
Total (estimate)	1462
Limit MEC-002 / margin	1800 / 19%

7.2 Power and endurance

Boat load	W
Flight controller + GNSS	0.8
Pi Zero 2W + camera + Wi-Fi (average)	1.8
LED beacon	0.3
ESC idle (2)	0.2
Motors at cruise (estimate)	8.0
Total at cruise	11.1
Usable energy: 3S 18650, 3.0 Ah, 80% usable	25.9 Wh
Estimated endurance (PWR-009 needs ≥ 40 min)	≈ 140 min
Bank load	W
Raspberry Pi 5 (average; peaks ~ 8 W during STT)	5.0
USB Wi-Fi adapter	1.0
Speaker, mic, LEDs	0.5
Total	6.5
20,000 mAh power bank (≈ 55 Wh delivered) $\rightarrow$ runtime (MC-001 needs ≥ 2.5 h)	≈ 8 h
Motor power is the dominant unknown. It is measured on the bench and in the pool before the endurance claim is verified.	

7.3 Cost

Where	Item	£
Boat	F405-class flight controller	25
Boat	M10 GNSS + compass	14
Boat	Pi Zero 2W	15
Boat	5 MP camera (OV5647-class) + Zero cable	8
Boat	microSD 32 GB	4
Boat	2 × brushless motor	12
Boat	2 × bidirectional ESC	10
Boat	3 × 18650 + 3S BMS	15
Boat	Box, PG7 glands, fuse	6
Boat	Foam, hi-vis, LED beacon	5
Boat	5 V 3 A buck	3
Boat	Reed switch + MOSFET switch module (arming key)	4
Bank	3 arcade buttons, key switch	7
Bank	USB mic + small speaker	8
Bank	USB Wi-Fi adapter with antenna	12
Bank*	2 m pole + 3 m USB extension (only if V-08 needs it)	8
	Baseline (boat + bank, excluding *)	148
	Boat only / bank only	121 / 27
	If the pole kit (*) is needed	156
	SRS cap (CON-001, Issue C) / target	160 / 150

CR-01 accepted (Issue B): the CON-001 cap is now £160, target £150 (SRS Issue C). The baseline of £148, or £156 with the pole kit, fits. As originally raised: the baseline was over the old £120 cap. Compared with the concept estimate (£117 including the Mission Control extras), the growth comes from five decisions:

- Python on the boat (Pi Zero instead of ESP32): +£9
- Bank radio (USB adapter + antenna): +£12
- Independent motor-power interlock: +£4
- Dedicated 5 V supply: +£3
- Adult key switch on the panel: +£3

The cheapest candidate, AR-3, would fit the cap, but it gives up the live photo pipeline. **Owner decision CR-01:** (a) raise the CON-001 cap to £160, target £150; or (b) keep £120 and adopt AR-3's action-camera photo path. **Owner chose (a).**

8. Design decisions register

ID	Decision	Basis	Effect
DD-01	Architecture AR-2 'Smart bank'	Section 3.3	-
DD-02	ArduPilot Rover on an F405-class flight controller, subject to V-01	T3	-
DD-03	Pi Zero 2W + 5 MP camera as the mission computer	T4	Resolves SWE-001 conflict
DD-04	LLM returns an intent; the Python planner generates the geometry	T1	-
DD-05	Missions run natively in ArduPilot AUTO; no streamed GUIDED control	Section 4.8	Keeps FS-007
DD-06	Pi 5 is the Wi-Fi AP (USB adapter, pole if needed); phone USB tether for internet	T2	Closes TBD-04
DD-07	On-device speech-to-text and text-to-speech on the Pi 5	T6	Closes TBD-05
DD-08	Web UI on phone or tablet as the display	T9	Closes TBD-06
DD-09	3S Li-ion 18650 pack with BMS	T7	Closes TBD-07
DD-10	Magnetic arming key cutting motor power; helm senses the rail	T8	-
DD-11	LED beacon driven by the helm, not the mission computer	Section 4.2	Works with MCP dead
DD-12	SRS modes mapped to ArduPilot modes as in section 4.6	Section 4.6	-
DD-13	Behaviours needing Lua are done in Python on the MCP	T3	FS-003/006/010
DD-14	Printed foam-filled hull segments; PVC pontoons as fallback	T5	-

Updated SRS open items: TBD-04, TBD-05, TBD-06 and TBD-07 are closed by DD-06 to DD-09. TBD-02 (which lake) and TBD-03 (the Trust's permission) remain open.

9. Assumptions to prove early, and risks

The architecture rests on a few assumptions about ArduPilot and the hardware. They are cheap to check in SITL or on the bench, so they're checked **first**, before most parts are bought or code is written.

ID	Assumption to confirm	Where	Protects
V-01	Chosen F405 board's ArduPilot Rover firmware includes: polygon fence with exclusion zones, battery, GCS and EKF failsafes, crash check, notify-LED output, second voltage input.	Bench	DD-02
V-02	Fence is enforced in the mode used for MANUAL.	SITL	FEN-004
V-03	GCS failsafe can 'continue in AUTO', with a 2 s timeout.	SITL	FS-002/003
V-04	Boat loiters at home at the end of RTL.	SITL	MOD-005
V-05	Crash check thresholds can meet FS-005.	SITL + pool	FS-005
V-06	Helm counts only Mission Control heartbeats as GCS (system IDs).	SITL	FS-002/003
V-07	On-device STT $\leq$ 5 s for a 5 s utterance; usable accuracy on a child's voice.	Bench	NLI-002
V-08	Link $\geq$ 100 m over water; with and without the pole.	Lake	COM-002
V-09	ESCs stop motors $\leq$ 1 s after signal loss.	Bench	FS-008
V-10	Pi Zero 2W in the sealed box at 30 °C for 60 min without throttling.	Bench	PWR-010
V-11	In GUIDED, the helm stops within 3 s if velocity targets stop arriving, so a B5 crash mid-burst is safe.	SITL	FS-006
V-12	A skid-steer boat frame is available in SITL and behaves plausibly.	SITL	SWE-004
V-13	The second voltage input can gate arming on motor-rail voltage (key in).	SITL + bench	MOD-003

ID	Risk	Level	Mitigation
R-01	F405 firmware lacks a needed feature	Medium	V-01 before purchase of other parts; H743 fallback (+£20-25)

ID	Risk	Level	Mitigation
R-02	Bill of materials over the £160 cap (was High at £120; CR-01 accepted)	Low	£12 headroom on baseline; pole kit only if V-08 needs it
R-03	Wi-Fi range over water	Medium	Pole antenna; safety independent of the link
R-04	Pi Zero SD-card corruption on power loss	Medium	Read-only root filesystem; photos on a separate partition; clean shutdown on disarm
R-05	Speech recognition struggles with a 4-year-old	Medium	Adult approval, templates, a short phrase set, typed fallback
R-06	LLM intent variability	Low	Schema-constrained output, validator, retries, templates
R-07	Weed	High	Guards, stuck detection, B5, site choice

10. Decisions and next documents

#	Decision	Status
1	Accept AR-2 'Smart bank' as the Mk1 architecture	Accepted (owner, 28 Sep 2026)
2	CR-01: cost cap (section 7.3)	Accepted: cap £160, target £150
3	Pi Zero 2W instead of ESP32 on the boat (DD-03)	Accepted as part of AR-2
4	Order of work: prove V-01 to V-06 in SITL and on the bench before the main parts order	Recommended; not yet confirmed

Then, from this document:

Document	Content	Source here
BOATY-ICD-001	One section per interface IF-01 to IF-22: definitions, timing, errors, verification	Section 5
BOATY-SSS-HUL/PRP/PWR	Mechanical and electrical subsystem specs	Appendix A, sections 4.2 and 4.5
BOATY-SSS-HLM	Helm spec: controlled ArduPilot parameter set, mode and failsafe config, verification	Sections 4.6 and 4.7
BOATY-SSS-MCP / MCN	Software and hardware specs for both computers	Section 4.3, Appendix A
BOATY-SSS-SIM	Simulation and test environment	SWE-004/005, section 9

Appendix A. Requirements by subsystem

HUL Hull & structure: primary 19, supporting 5

Primary: CAM-006, REC-001, MEC-001, MEC-003, MEC-004, MEC-005, MEC-006, MEC-007, MEC-009, MEC-012, MEC-014, MEC-015, ENV-003, ENV-004, ENV-006, CHD-001, CHD-002, CHD-003, CHD-004

Supporting: CAM-005, MEC-008, MEC-013, PWR-010, ENV-002

PRP Propulsion: primary 7, supporting 8

Primary: NAV-001, FS-008, MEC-008, MEC-010, MEC-011, ENV-002, ENV-005

Supporting: NAV-002, NAV-003, FS-006, PWR-009, CHD-001, CHD-002, CHD-003, CHD-004

PWR Power: primary 10, supporting 7

Primary: MOD-003, PWR-001, PWR-002, PWR-003, PWR-004, PWR-005, PWR-006, PWR-007, PWR-008, PWR-011

Supporting: MOD-002, PRE-003, PRE-007, MEC-012, PWR-009, ENV-004, CHD-003

HLM Helm: primary 37, supporting 22

Primary: MOD-001, MOD-002, MOD-004, MOD-005, MOD-008, PRE-001, PRE-002, PRE-003, PRE-004, PRE-007, PRE-008, NAV-002, NAV-003, NAV-004, NAV-005, NAV-006, NAV-007, NAV-008, FEN-001, FEN-004, FEN-005, FEN-006, FEN-007, FS-001, FS-002, FS-003, FS-004, FS-005, FS-007, FS-011, FS-012, SAF-001, SAF-002, SAF-003, COM-007, MEC-013, LOG-001

Supporting: MOD-003, MOD-006, PRE-005, NAV-001, FS-008, FS-009, FS-010, SAF-007, MIS-001, MIS-002, MIS-003, MIS-004, MIS-005, MIS-006, DET-004, MC-014, REC-004, PWR-005, PWR-006, PWR-010, SWE-008, ENV-002

MCP Mission computer: primary 12, supporting 14

Primary: FS-006, FS-010, CAM-001, CAM-002, CAM-003, CAM-004, CAM-005, CAM-007, CAM-008, DET-003, DET-004, PWR-010

Supporting: PRE-008, FS-003, FS-007, CAM-006, COM-001, COM-002, COM-003, COM-004, COM-006, PWR-005, SWE-004, SWE-008, LOG-004, ENV-005

MCN Mission Control: primary 65, supporting 22

Primary: MOD-006, MOD-007, PRE-005, PRE-006, FEN-002, FEN-003, FS-009, SAF-004, SAF-007, MIS-001, MIS-002, MIS-003, MIS-004, MIS-005, MIS-006, NLI-001, NLI-002, NLI-003, NLI-004, NLI-005, NLI-006, NLI-007, NLI-008, VAL-001, VAL-002, VAL-003, VAL-004, VAL-005, VAL-006, VAL-007, VAL-008, VAL-009, VAL-010, DET-001, DET-002, MC-001, MC-002, MC-003, MC-004, MC-005, MC-006, MC-007, MC-008, MC-009, MC-010, MC-011, MC-012, MC-013, MC-015, MC-014, COM-001, COM-002, COM-003, COM-004, COM-005, COM-006, SWE-002, SWE-003, SWE-008, SWE-009, LOG-002, LOG-003, LOG-004, LOG-005, CHD-005

Supporting: MOD-001, MOD-003, MOD-004, MOD-005, PRE-001, PRE-002, PRE-004, PRE-008, FEN-001, FEN-004, FEN-005, FEN-006, FEN-007, FS-012, SAF-001, SAF-002, SAF-003, CAM-007, CAM-008, SWE-004, OPS-001, CON-002

REC Recovery & signalling: primary 6, supporting 0

Primary: REC-002, REC-003, REC-004, REC-005, REC-006, REC-007

SIM Simulation & test: primary 3, supporting 0

Primary: SWE-004, SWE-005, SWE-007

SYS System level: primary 12, supporting 0

Primary: FS-013, SAF-006, MEC-002, PWR-009, SWE-001, SWE-006, ENV-001, CHD-006, CON-001, CON-002, CON-003, CON-004

OPS Operations: primary 12, supporting 1

Primary: OPS-001, OPS-002, OPS-003, OPS-004, OPS-005, OPS-006, OPS-007, OPS-008, OPS-009, OPS-010, OPS-011, OPS-012

Supporting: PRE-006

Not applicable: SAF-005